

WEEK 4 ASSIGNMENT REPORT

Azure Cloud Fundamentals and Data Pipeline Implementation using Azure Data Factory

Submitted by
Kashish Chadha
Data Engineering Intern

Student ID	CT_CSI_DE_1098
Stream	B.Tech (CSE)
College	DIT University
Batch	Batch 1
Passing Out Year	2027
Domain	Data Engineering
Program Start Date	15/06/2026

1. Objective

The goal of this assignment was to get hands-on with the core building blocks of Microsoft Azure — storage, resource organisation, and access management — and then tie them together into a working data pipeline using Azure Data Factory (ADF). Rather than just reading about these services, the idea was to actually provision them, break a few things, fix them, and end up with a pipeline that reliably moves and validates a CSV file from a Blob container to a destination location.

Specifically, the assignment covers:

- Creating and organising Azure resources inside a Resource Group
- Setting up a Storage Account and Blob Container, and uploading a source dataset
- Building an Azure Data Factory instance and exploring its Author / Monitor / Manage views
- Creating a Linked Service, source and destination Datasets
- Using a Get Metadata activity to validate the source file before copying it
- Building and running a Copy Data pipeline
- Assigning IAM roles so that ADF has the right level of access to the storage layer

2. Tools & Technologies Used

Cloud Platform	Microsoft Azure
Storage	Azure Blob Storage (Storage Account + Container)
Orchestration	Azure Data Factory (ADF) – Author, Monitor & Manage hubs
Access Control	Azure IAM (Role-Based Access Control)
Dataset	Sample Superstore CSV (Kaggle)

3. Implementation Walkthrough

This section documents each task in the order it was carried out, along with the reasoning behind key decisions and the screenshot captured as evidence for that step.

Task 1 — Resource Group

Every resource used in this assignment (storage account, ADF instance, and later the IAM assignments) needed to live inside a single Resource Group. Grouping them this way makes it much easier to track cost, manage permissions as a unit, and clean everything up in one shot once the assignment is done, instead of hunting down loose resources scattered across the subscription.

Home > CelebalAssignment > Marketplace > Storage account

Create a storage account

Select the subscription in which to create the new storage account. Choose a new or existing resource group to organize and manage your storage account together with other resources.

Subscription * Azure for Students

Resource group * CelebalAssignment
[Create new](#)

Instance details

Storage account name * mystorage

Region * (Asia Pacific) Central India
[Deploy to an Azure Extended Zone](#)

Primary service * Azure Blob Storage or Azure Data Lake Storage

i This helps us provide relevant guidance. It doesn't restrict your storage to this resource type. [Learn more](#)

Performance * ☒ Standard: Recommended for most scenarios (general-purpose v2 account)
☐ Premium: Recommended for scenarios that require low latency.

Redundancy * Locally redundant storage (LRS)

Previous Next Review + create

Figure 1: Resource Group created for the assignment

Task 2 — Storage Account & Blob Container

Next, a Storage Account was provisioned inside the same resource group. Inside it, a Blob Container was created to act as the landing zone for raw data. The Superstore sample dataset (CSV) was then uploaded into this container — this is the file that the pipeline reads from in later steps.

Home > stadfweek4lab02_1783735847496 | Overview > stadfweek4lab02

CelebalAssignment
Resource group

Search < > + Create Manage view Delete resource group Refresh Export to CSV Open query Assign tags Move Delete

Overview

- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Events
- Settings
- Cost Management
- Monitoring
- Automation
- Help

Essentials

Resources Recommendations

Filter for any field... Type equals all Location equals all Add filter

Name ↑	Type	Location
stadfweek4lab02	Storage account	Korea Central

Showing 1 - 1 of 1. Display count: auto

Add or remove favorites by pressing Ctrl+Shift+F

Figure 2: Storage Account provisioned for the pipeline

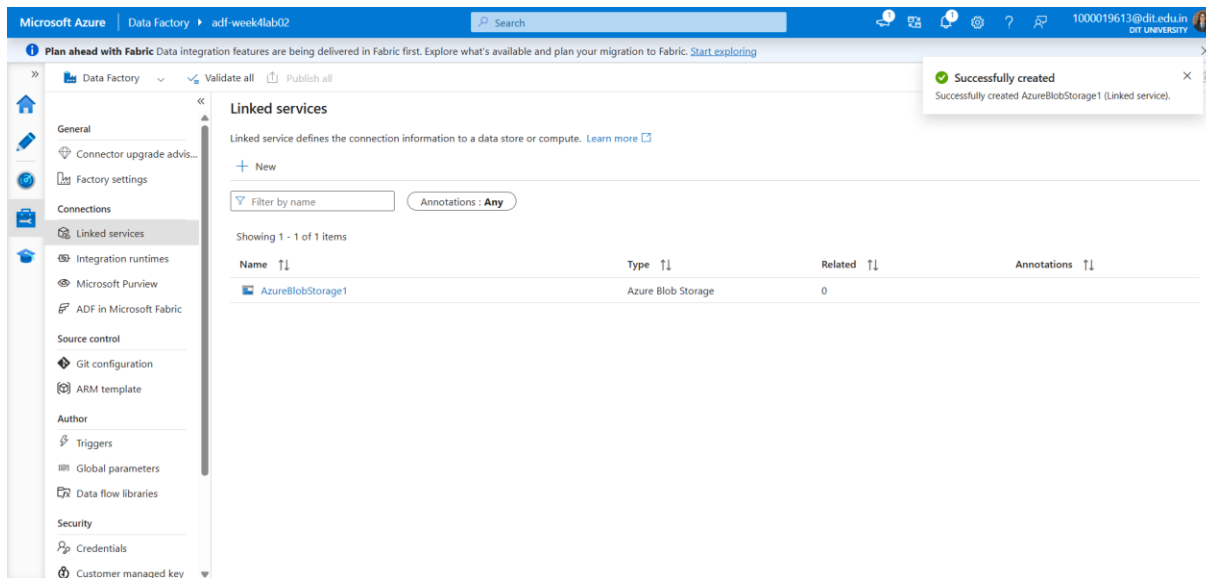


Figure 3: Blob Container with the uploaded CSV file

Task 3 — Azure Data Factory Basics

With the storage layer ready, an Azure Data Factory instance (adf-week4lab02) was created. Spending time in the Author, Monitor, and Manage hubs helped in understanding how ADF separates pipeline design, run history, and connection management into distinct areas instead of cramming everything into one screen.

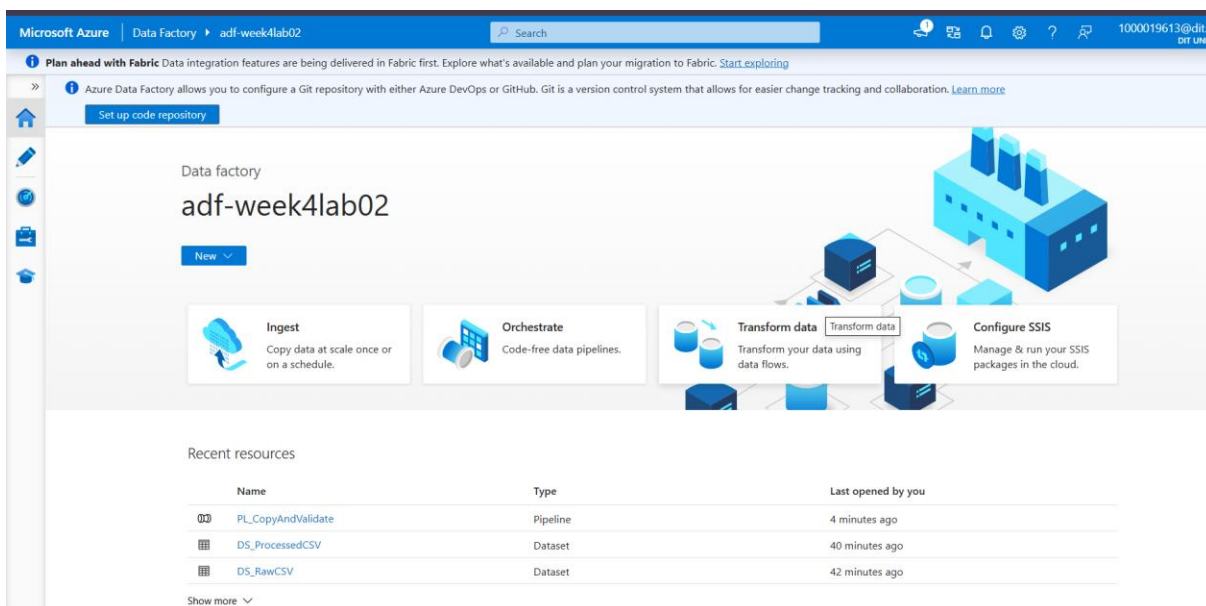


Figure 4: Azure Data Factory instance overview

A Linked Service was then configured to connect ADF to the Blob Storage account — this is essentially the authenticated connection string ADF uses to reach the storage account.

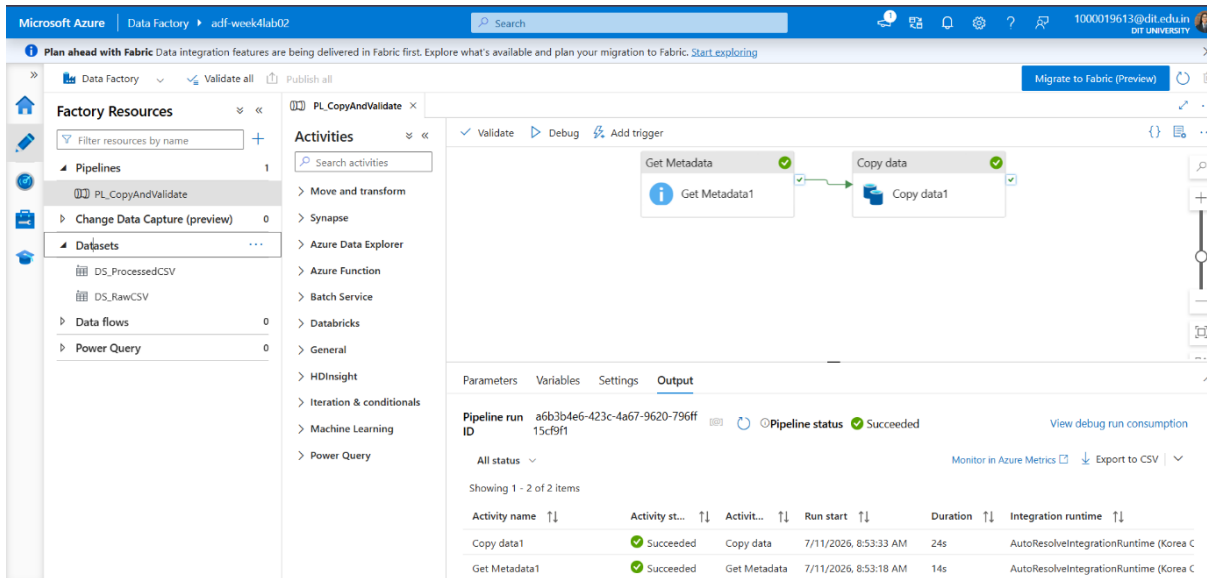


Figure 5: Linked Service connected to Blob Storage

Two datasets were created on top of this linked service: DS_RawCSV, pointing at the source file, and DS_ProcessedCSV, pointing at the destination location where the validated file would be copied to.

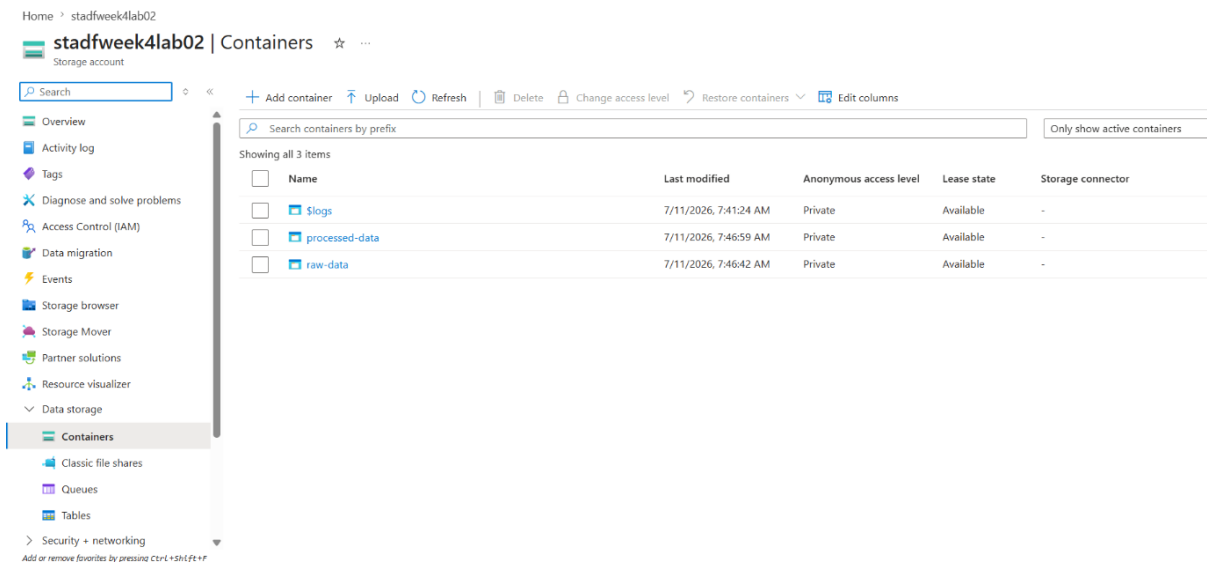


Figure 6: Source dataset (DS_RawCSV) configuration

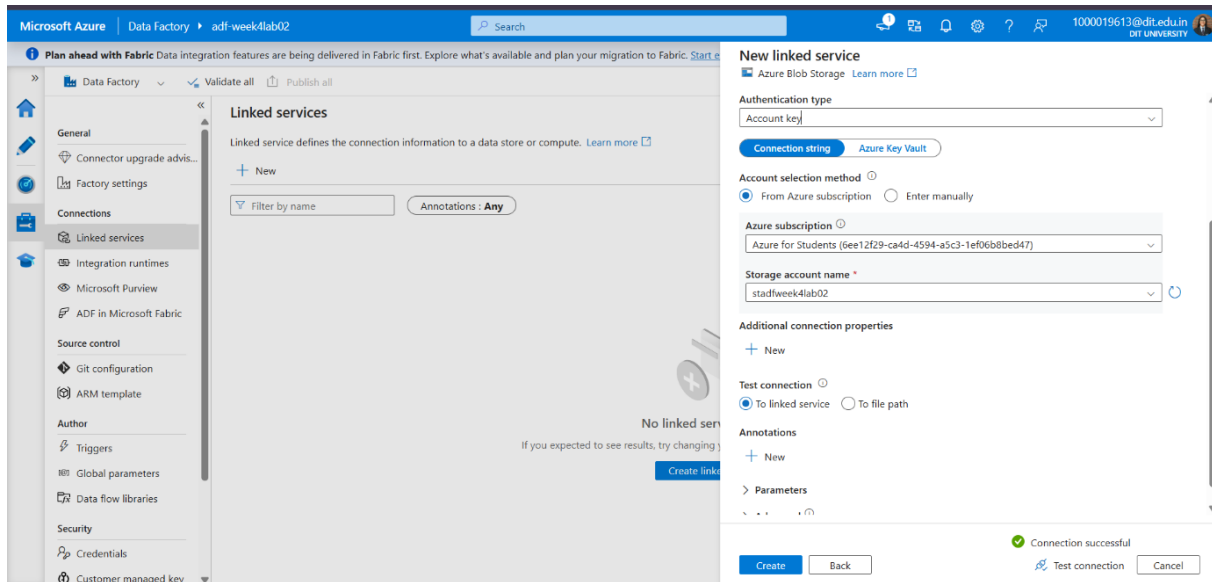


Figure 7: Destination dataset (DS_ProcessedCSV) configuration

Before copying anything, a Get Metadata activity was added and pointed at the source dataset. This activity checks properties of the file (such as its existence, size, and last-modified timestamp) before the pipeline commits to copying it — a simple but useful safety check.

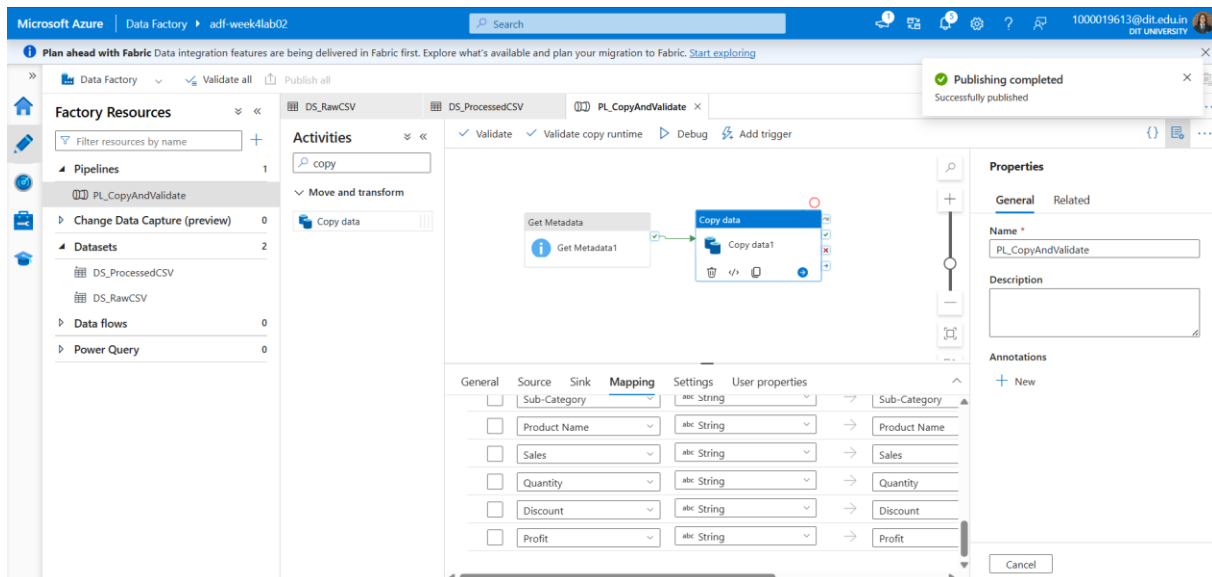


Figure 8: Get Metadata activity output showing file properties

Task 4 — Pipeline Development

The two activities were then wired together inside a single pipeline, PL_CopyAndValidate: Get Metadata1 runs first and, on success, triggers Copy data1, which copies the file from the source container to the destination path. Chaining them this way means the copy only happens once the source file has been confirmed to be valid and accessible.

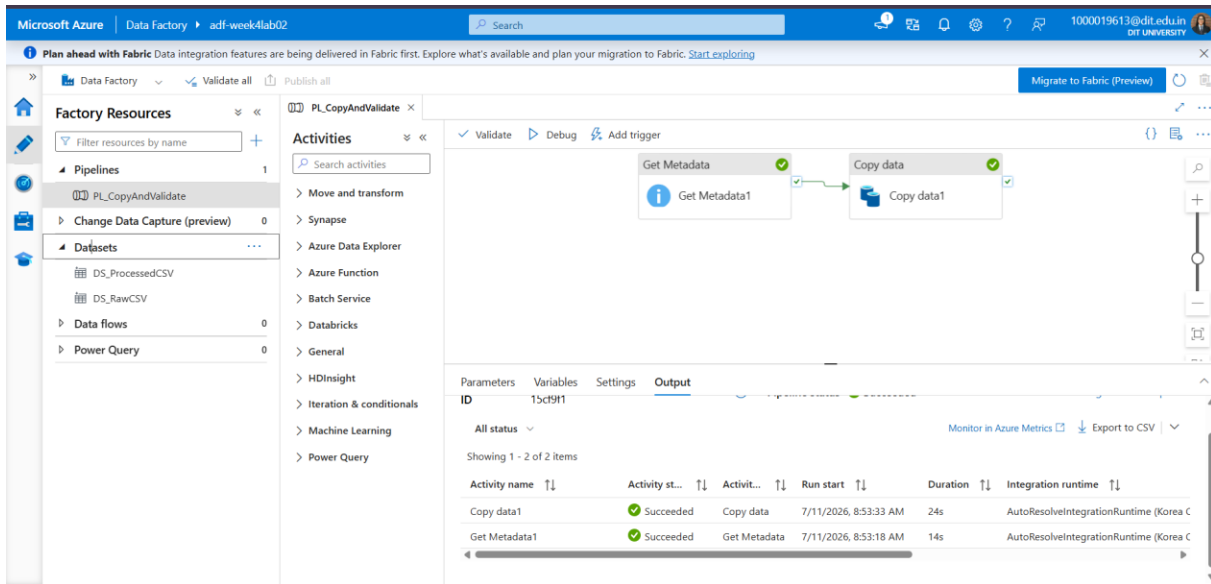


Figure 9: Pipeline design — Get Metadata → Copy Data

Task 5 — Pipeline Execution

The pipeline was run in Debug mode. Both activities completed successfully, and the Output tab confirmed a 'Succeeded' status for the full run, along with the duration and integration runtime used for each activity.

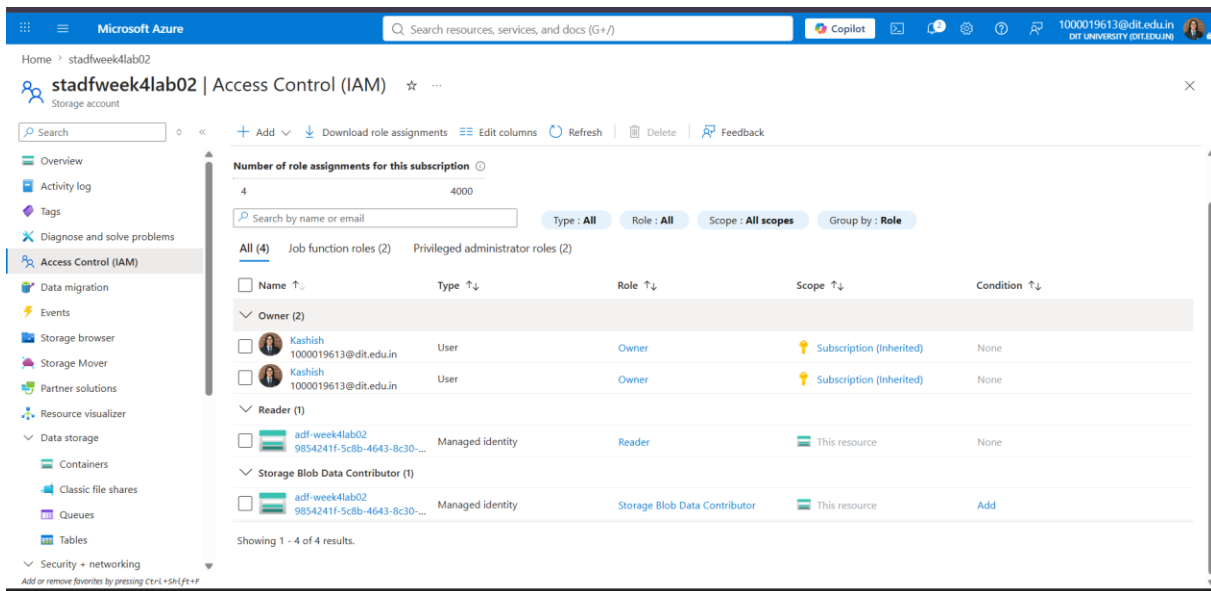


Figure 10: Pipeline run — both activities succeeded

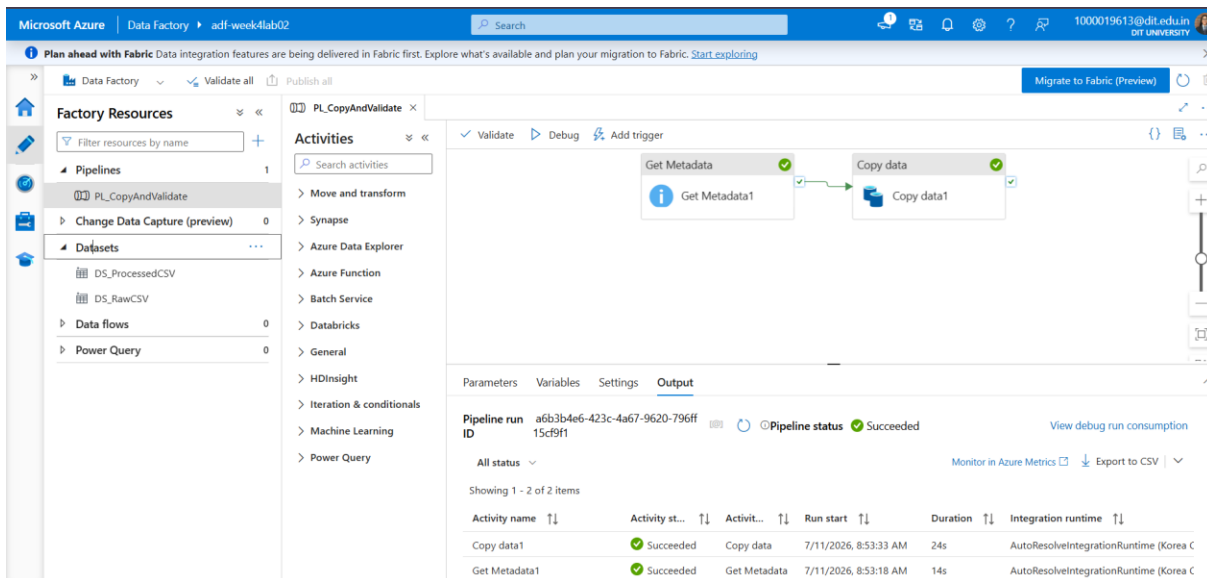


Figure 11: Detailed run output with activity durations and status

Task 6 — IAM Roles

To make sure ADF's access to the storage account followed least-privilege principles rather than relying on broad account-level access, two roles were assigned to the Data Factory's managed identity directly on the Storage Account, via Access Control (IAM):

- Reader — allows viewing the resource and its configuration
- Storage Blob Data Contributor — the actual role that permits ADF to read and write blob data, since the built-in 'Contributor' role only manages the resource shell and does not grant data-plane access

This distinction between control-plane and data-plane permissions was one of the more useful things to internalise from this task — it's a common gotcha when a pipeline works from the portal but a service principal or managed identity fails with an access-denied error at runtime.

4. Mini Project — End-to-End Validation

The mini project tied every earlier task together into one working flow: a CSV file sitting in Blob Storage is picked up by ADF, its metadata is validated with a Get Metadata activity, and it is then copied to a destination location using a Copy Data activity — all inside a single orchestrated pipeline (PL_CopyAndValidate). The successful Debug run (Figures 10–11) confirms that the metadata check passed and the file was copied to its destination without errors.

5. Key Learnings

- Resource Groups aren't just organisational sugar — they make IAM scoping, cost tracking, and teardown significantly simpler once more than one resource is involved.
- Linked Services and Datasets are deliberately separate concepts in ADF: a Linked Service is the connection, while a Dataset is a named, reusable pointer to a specific file or table within that connection.
- Get Metadata is a lightweight but effective way to fail a pipeline early if the source file is missing or malformed, instead of discovering that mid-copy.

- IAM in Azure has a control-plane vs data-plane distinction that isn't obvious at first — 'Contributor' on a Storage Account does not automatically grant permission to read or write blobs.
- Debug runs in ADF are invaluable for iterating quickly, since they surface per-activity duration and status without needing to publish or trigger a full pipeline run.

6. Challenges Faced

- Initially assigning only the 'Contributor' role and assuming that was sufficient for ADF to read blob data — this had to be corrected by adding 'Storage Blob Data Contributor' separately.
- Locating the IAM/Access Control settings, since they live on the Storage Account resource itself and not inside the Data Factory UI, which isn't immediately obvious when navigating ADF for the first time.
- Getting the Get Metadata activity's output structure right so it could be meaningfully chained before the Copy activity ran.

7. Conclusion

This assignment gave a practical, end-to-end feel for how a simple but realistic data pipeline is put together on Azure — from raw infrastructure (resource groups, storage) up through orchestration (ADF) and finally access control (IAM). The resulting pipeline, PL_CopyAndValidate, reliably validates a source file's metadata before copying it to its destination, and the underlying IAM setup ensures ADF only has the specific permissions it needs on the storage account. The full implementation, including all configuration screenshots, has also been documented and pushed to a GitHub repository for reference.